Group 3 - Space Complexity

Members:
Kristy Mok, Ben Chalk, Corina Rivero Montiel, Euan Faller, Joel Cutler, Luke Callen, Sam Ade Fowodu

Continuous Integration Report

6a. Continuous Integration Methods and Approaches

Our project uses a well-structured Continuous Integration approach to maintain seamless integration, uphold high-quality standards, and optimise development workflows. The following outlines the key components of our methodology and their suitability for the project:

1. **Branch-Based Workflow:**
   Development is organised around feature-specific branches. Once a feature has been implemented, a pull request is created. This facilitates rigorous code reviews by other team members, enabling the identification of bugs and ensuring adherence to coding standards. This process not only safeguards the quality of the main branch but also fosters collaboration and knowledge sharing within the development team.
2. **Incremental Commits:**
   Developers commit small, incremental changes throughout the development process to ensure traceability, stability and efficiency. Traceability from being able to easily find and isolate changes which simplifies debugging and issue resolution. Stability by allowing reversions to previous working states with minimal disruptions when necessary. Efficiency as developers can experiment and refine features without fear of jeopardising the project's overall progress.
3. **Automated Testing and Builds:**
   Using GitHub Actions, we have established an automated workflow that is triggered whenever a pull request is submitted or changes are pushed to the main branch. This includes an automated testing pipeline incorporating unit tests, code style checks, and coverage analysis to provide comprehensive quality assurance. This ensures that developers can catch errors and receive immediate feedback regarding integration issues before they reach the main branch. Furthermore, the pipeline includes an automated build process which can guarantee consistent builds and reduces the likelihood of discrepancies between local and shared environments.
4. **GitHub Projects:**
   GitHub Projects serves as a centralised platform for task management. Tickets are created for individual features or issues, assigned to developers, and tracked through various stages of development. This system ensures transparency, enhances accountability, and helps the team stay aligned with project milestones and priorities.
5. **Gradle and Java-Specific Configuration:**
   Gradle, a versatile build automation tool, is utilised to manage dependencies and streamline the build process. Its efficiency and flexibility enables us to achieve fast, reliable builds tailored to the project's requirements.
6. **Future Plans:**
   In conclusion, our CI approach adheres to industry best practices, promoting collaboration, maintaining high code standards, and facilitating continuous improvement as the project evolves. The scalability and reliability of this methodology ensure that it will continue to support the development team effectively.

6b. Continuous Integration Infrastructure Implementation

The Continuous Integration (CI) infrastructure for our project leverages GitHub Actions, providing a robust framework for automating builds, tests, and code analysis. Below is an in-depth explanation of its implementation:

1. **Trigger Points:**
   The CI workflows are configured to activate under the following conditions:
   - **Push Events:** When changes are pushed to the main branch, the pipeline is triggered to build the project and verify the changes integrate successfully.
   - **Pull Requests:** When a pull request targeting the main branch is created, the pipeline ensures that the proposed changes meet the project's quality and compatibility standards. This is essential for keeping the integrity of the codebase and minimising disruption to ongoing development.
2. **Pipeline Overview:**
   The pipeline oversees several critical tasks, including:
   - Checking out the repository to access the latest code.
   - Setting up the Java development environment and Gradle.
   - Running Checkstyle to enforce code style guidelines.
   - Executing unit tests and generating reports.
   - Generating and uploading JaCoCo coverage reports.
   - Building the project and uploading the executable JAR file.

   If any tests fail the pipeline will fail giving clear feedback that changes are required.
3. **Implementation Details:**
   The CI pipeline is defined in the .github/workflows directory as a YAML configuration file. Below is the implementation of some key steps included in the pipeline:
   - **Checkout and Setup:** The repository is checked out using actions/checkout@v4, ensuring the latest code is accessible. The Java Development Kit (JDK 17) is set up using actions/setup-java@v4, and Gradle is configured with gradle/actions/setup-gradle@v4.
   - **Code Style Checks:** Checkstyle tasks, including 'checkstyleMain' and 'checkstyleTest', are executed to ensure that the source code for the game and tests adheres to googles style guidelines. Reports are generated, collected together and uploaded, providing actionable insights for developers to address style violations.
   - **Unit Tests and Reports:** Our unit tests are executed, with the reports being uploaded, enabling developers to verify the correctness of new features and prevent regressions. Jacoco is used to provide a coverage report for these tests, which is also uploaded allowing developers to guarantee our tests are covering the majority of the codebase.
   - **Build:** The project is built using Gradle after all checks and tests pass successfully. The built JAR file is found and uploaded. This step ensures that the final output is dependable and ready for deployment or distribution.
4. **Future Plans:** The pipeline is designed with scalability in mind, allowing for the integration of other testing frameworks, deployment processes, and performance monitoring tools as the project evolves. These enhancements will ensure that the CI infrastructure continues to meet the demands of a growing codebase and team.

By using GitHub Actions and Gradle, our CI infrastructure delivers a comprehensive, scalable solution for maintaining code quality and streamlining the development process. This implementation not only supports current project needs but also positions the team for long-term success.